

Contract-First APIs in the Age of AI Agents

“Because your API now has a consumer who never reads the docs.”



ERROR

status: **Action!**
error: Something went wrong!

Status: **Oops!**

traceID: xyz

ERROR

ERROR

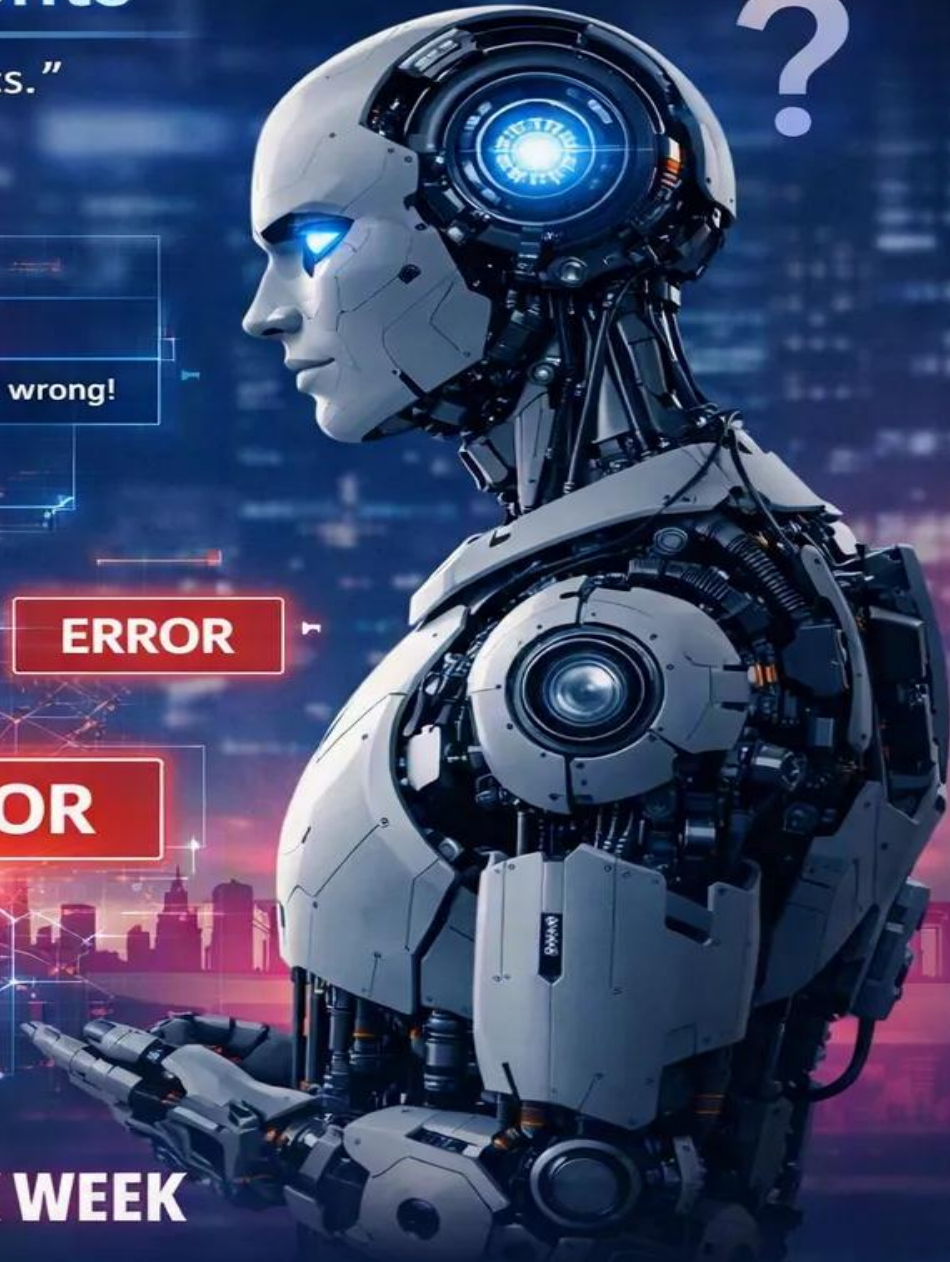


API
CONFERENCE



devm.io

NEW YORK WEEK



Meet **Brittany**

- Mid-level developer at a **professional services firm**.
- Frequent flyer — **Delta Diamond Medallion** level.
- Built her own **AI travel agent**: calls Delta's API, monitors itinerary, rebooks flights when plans change, sends Slack alerts.
- Deployed it **last Thursday**. First trip with it live: **Monday**.

"She trusts the agent. The agent trusts the API. Let's see how that goes."



BRITTANY

Senior Platform Engineer

 Diamond Medallion

 Agent Deployed

Your API Now Has Two Kinds of Consumers



Human Developer

Reads between the lines

- Reads the documentation
- Interprets ambiguity
- Debugs and adapts
- Asks questions in forums
- Works around inconsistencies



AI Agent

Executes literally – no exceptions

- Reads schemas
- Executes deterministically
- **Cannot adapt to ambiguity**
- **Retries blindly on failure**
- **Repeats mistakes at scale**

"Ambiguity that a human navigates is an outage that an agent creates."

When Ambiguity Meets Automation

Failure chain — every step behaved as designed. The contract was the thing that failed.

Failure Cascade — Monday, 5:45 AM

- 1 **Network timeout.** POST /bookings goes out; no response comes back.
- 2 **The agent retries — correctly.** No idempotency key, so the second POST creates a second booking.
- 3 **status: "ok" on both.** Success or pending? The agent stores one confirmation; two reservations exist.
- 4 **Seat assignment fails:** “Something went wrong. Please try again.” No code, no retry guidance.
- 5 **Six retries, then it gives up** — and marks the booking confirmed anyway.

5:45 AM

Brittany is at the airline counter. Two boarding passes. Neither seat is confirmed.

Her agent behaved exactly as designed.

The API contract was what failed her.

The Contract Is the Truth

Rewind — Build the Fix

Core Definition

Contract-First API Design

- You write the specification **before** you write the code.
- The **spec comes first**. The code fulfills the spec.
- The OpenAPI spec is the **source of truth** — not generated output of your code, not documentation updated after the fact.

What a Real Contract Includes

- ✓ **Request & response shapes**
with exact types
- ✓ **Enum values**
not arbitrary strings
- ✓ **Error formats**
machine-readable codes, not freeform messages
- ✓ **Retry semantics**
what is retryable; how long to wait
- ✓ **Versioning policy**
changes communicated & deprecated
- ✓ **Observability hooks**
trace info on every response

Who Reads the Contract

- 👤 The consuming developer
- ⚙️ The CI pipeline
- 📄 The SDK generator
- 🛡️ The API gateway
- 📖 The documentation site
- 🤖 **And now: the AI agent.**

*"Design rules that live in a wiki get ignored. Design rules **enforced at the gateway** get followed."*

Determinism Is a Feature

Principle 1: Make the Happy Path Boring

✗ Before — Ambiguous

```
status : "ok"  
result : "done"  
// What does "ok" mean?  
// The agent cannot know.
```

✓ After — Contract-First

```
status : "CONFIRMED" | "PENDING" | "FAILED"  
  
booking_state : {  
  status : enum ,  
  updated_at : ISO8601 ,  
  confirmation_number : string | null  
}
```

Determinism Checklist

- ✓ **Enums for every status field** — closed set of known values, documented in schema, enforced by contract
- ✓ **Stable keys** — add fields across versions, never rename or remove
- ✓ **Explicit nullability** — if a field can be absent, say so in the schema
- ✓ **Correlation ID on every response** — not just errors
- ✓ **Consistent pagination** — if your cursor field changes name by endpoint, agents break silently

Principle 1 Payoff

With a proper status enum, the agent knows the booking is PENDING and waits for CONFIRMED — rather than double-booking. "Brittany still has one boarding pass."

Your Error Is an Agent's Next Step

Principle 2

02

Principle

Structured Error Schema

```
{
  "error_code": "SEAT_UNAVAILABLE",
  "message": "The selected seat is no longer available.",
  "retryable": true,
  "retry_after": 30,
  "remediation": "Query /flights?origin=ORD&dest=ATL&flexible=true"
}
```

Field Design Rules

error_code Machine-readable, consistent, stable string

message Human-readable description

retryable Boolean; if **true**, agent retries; if **false**, agent escalates and stops

retry_after If retryable and wait time known, include it (seconds)

remediation Optional but powerful — a next-step hint for the agent

Agent Decision Tree

 **Receive error response** → **check retryable**

retryable: true

retryable: false

 **Wait** retry_after seconds

 **Surface to user** immediately

 **Retry once** using remediation hint if present

 **Stop** — do not retry

 **Escalate** if retry also fails

✓ **IRROPS payoff:** Apex's API returns a structured error. Agent finds next available flight, books it, sends Slack message.
"That is what actionable errors unlock."

Agents Will Retry. Make It Safe.

Principle 3: Idempotency

03

Principle

Idempotency-Key — Request Flow

```
// 1. Client generates UUID before the request
Idempotency-Key: "550e8400-e29b-41d4-a716-446655440000"

// 2. POST to booking endpoint — includes the key
POST /bookings
  Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

// 3. Network timeout. Agent retries with SAME key.
//   Server recognizes key — returns original confirmation.
//   No second booking. No second charge.

// 4. Contract documents the guarantee:
//   "This endpoint is idempotent when
//   Idempotency-Key is provided.
//   Clients should generate a UUID per
//   request and reuse on retry."

POST /bookings ← retry with same key
→ 200 OK { "confirmation": "BK-99182", "duplicate": false }
```

What happens with Idempotency-Key

-  **Agent POSTs /bookings**
UUID key generated & sent in header
-  **Network timeout**
No response received — agent retries
-  **Retry with same UUID key**
Server recognizes key — returns original result
-  **One booking. One charge.**
No duplicate. No second reservation.

Which endpoints get Idempotency-Key first

- ✓ Any endpoint that **charges money**
- ✓ Any endpoint that **sends notifications**
- ✓ Any endpoint that **triggers external side effects**
- ✓ Any **POST, PUT, or PATCH** that creates or modifies state

"If it's not in the contract, the agent won't use it. And if the agent doesn't use it, you will have Brittanys."

 **Resolution:** Same Saturday night timeout. Same retry. But this time — Idempotency-Key on the request. Retry uses same key. **One boarding pass. The duplicate booking never happened.**

Trace IDs Are Not Optional

Principle 4

Principle
04

correlation_id

Unique to the end-to-end transaction. Shared across ALL calls in a single agent workflow. Connects the dots across a multi-hop chain.

request_id

Unique to that specific individual call. Used to find the exact call in logs, traces, and support tickets.

Agent Workflow — Flight Cancellation Rebooking

Search /flights

corr_id: abc-123
req_id: req-001



Select /seats

corr_id: abc-123
req_id: req-002



POST /bookings

corr_id: abc-123
req_id: req-003



GET /bookings/{id}

corr_id: abc-123
req_id: req-004

✓ **One correlation_id** → full audit trail in **5 minutes**

✗ **No correlation_id** → grep logs by timestamp + user ID → **40-minute hold queue**

W3C Trace Context

The **traceparent** header — interoperable with every major observability platform.

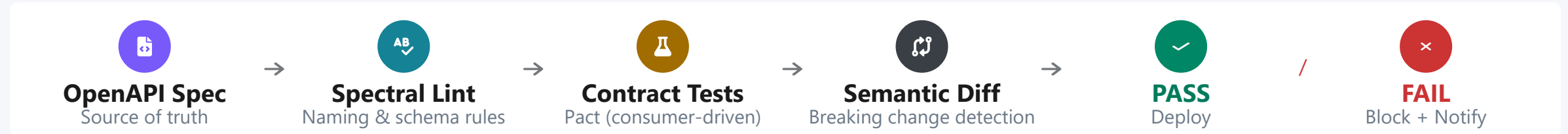
*"Trace IDs aren't just nice to have. They're the difference between a **five-minute audit** and a **forty-minute hold queue**."*

Contracts Without Governance Are Just Documents

Governance
CI Pipeline

Breaking change caught in CI pipeline before reaching Brittany's agent.

Governance Pipeline



Consumer-Driven Contract Testing

- Consumer defines exactly what it expects — fields, types, status values
- That definition becomes a **test**
- Provider runs it in CI **before every deploy**
- If the change breaks expectations → **pipeline fails**

Framework: **Pact** — pact.io

Versioning Policy

- **Semantic versioning** for APIs
- Deprecation headers in responses
- Sunset dates documented in the spec
- Both teams accountable to the contract

"Contracts have tests. Tests have pipelines. Pipelines have opinions."

Governance Payoff: breaking change caught in CI, not production

Four emerging patterns that make your API agent-ready today — and agent-discoverable tomorrow

01 Write Descriptions for Agents

OpenAPI **description** fields are read by LLMs for tool selection. Write them like explaining to an intelligent but literal colleague.

- What this endpoint does and *when* to call it
- What key parameters mean
- What the caller should do with the response

"One good paragraph. Write it."

02 Model Context Protocol (MCP)

Emerging standard for exposing APIs as **discoverable, composable tools** for AI agents.

MCP

OpenAPI spec
→ **MCP manifest**
→ Agent tool surface

The contract work you do *today* becomes agent-discoverability *tomorrow*.

03 Async Job Pattern

Long-running operations must not block agents synchronously.

POST /bookings → 202 Accepted
{ job_id, status: PENDING, poll_url }

↓

GET /jobs/{id} → 200 OK
{ status: CONFIRMED, booking_id }

Agent polls or subscribes to webhook — no blocking, no timeouts.

04 Per-Agent Identity

Each agent gets its own identity — separate from the human user.

OAuth 2.1 + scoped tokens

- Auditable independently from its human owner
- Revocable without affecting the user account
- Scoped to exactly what the agent needs — no more



*Agent discovers Apex's full tool surface from the MCP manifest. A new endpoint is added, and the **agent picks it up automatically**. "The contract is the API. The API is the agent tool. They're the same thing now."*

Brittany Lands — On Time

Resolution Map — Act I Failures vs. Contract-First Fixes

Full Arc Close

What Went Wrong

False Success

status: "ok" — agent cannot distinguish confirmed from pending, double-booked

Duplicate Charge

No idempotency key — retry created a second booking and second charge

Retry Loop ×6

Unstructured error: "Something went wrong" — agent retried 6 times, gave up

No Audit Trail

No trace IDs — cannot debug multi-hop chain; 40-minute hold queue

Breaking Change

Breaking change reached production undetected — no contract enforcement

Contract-First Fix

Principle 1

Enums + deterministic schemas: CONFIRMED | PENDING | FAILED

Principle 3

Idempotency-Key header documented in contract — retry reuses same key

Principle 2

Actionable error schema with retryable + remediation fields

Principle 4

correlation_id + request_id on every response

Governance

Consumer-driven contract testing in CI/CD — caught before deploy

Delta Air Lines — Boarding Pass

ORD

→

ATL

Passenger

Seat

Class

B. Shaw

5A

First

★ Upgraded via MCP manifest

"Monday morning. Brittany walks to the kiosk. One boarding pass. Seat 5A — she got upgraded. Her agent noticed her AirMiles status and pinged the upgrade waitlist endpoint, documented in the MCP manifest. She didn't change her travel habits. She didn't rebuild her agent. Apex changed their API contract. That was enough."

— Full Arc Close



Start With the Contract. Ship the Code Second.

Call to Action

✓ Audit one API for agent-readiness

Pick the most-called or highest-stakes endpoint. Ask: enums on every status field? Correlation ID? Idempotency on side-effecting POSTs?

✓ Write OpenAPI first for your next endpoint

Just the next one. Before a single line of code. Write the spec, get a teammate to review it.

✓ Add idempotency keys to your highest-risk endpoint

Pick the one that charges money or sends notifications. The implementation is a middleware layer. The value is immediate.

✓ Set up one consumer-driven contract test in CI

One Pact test. One. It will change your team's culture in ways a style guide never will — because it has teeth.

✓ Rewrite your description fields

Three most-called endpoints. Write them as if an intelligent but literal system will read them. Because one is.

Closing Reframe

*"Brittany is not a fictional character. Brittany is **your users. Right now.** Someone is building an AI agent to interact with your API. Their agent is calling your API today. The question is whether your contract is ready for it."*

Monday Morning

Five concrete steps. Any one of them ships value. Start with the one that scares you most.

Questions Welcome

Especially if your Brittany story is worse than this one.

Resources from this talk



OpenAPI Specification

openapis.org — contract tooling, start here



Pact Framework

pact.io



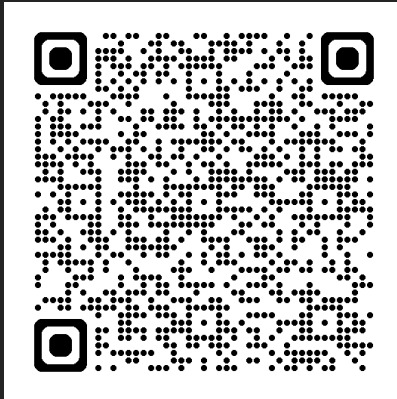
W3C Trace Context

w3.org/TR/trace-context



MCP Specification

modelcontextprotocol.io



Link to slides, resources,
and schemas

Chad Green

Senior Systems Architect
Jasper Engines & Transmissions

 chadgreen@chadgreen.com

 <https://www.chadgreen.com>

 [chadwickgreen](https://www.linkedin.com/in/chadwickgreen)

 [@chadgreen](https://twitter.com/chadgreen)

 [TaleLearnCode](https://github.com/TaleLearnCode)